



古诗转水墨画

——AI 课程设计作业 4

实验报告

计算机科学与技术学院

2025 年 6 月 10 日

AI 课程设计作业 4——古诗转水墨画（大模型）

1.问题概述

(1)问题的直观描述

①实验内容

古诗与水墨画作为中华优秀传统文化的瑰宝，蕴含着深厚的文化底蕴与艺术价值。古诗以凝练的语言描绘出丰富的意境，水墨画则通过笔墨勾勒出独特的视觉美感。将二者融合，借助深度学习技术实现古诗意境的可视化表达，具有重要的文化传承与艺术创新意义。

Stable Diffusion（SD）模型作为当前先进的潜在扩散模型，在通用图像生成任务中表现出色，但在处理古诗水墨画生成这类特定领域任务时，原生模型难以精准捕捉古诗中的文化内涵与水墨画的艺术风格。因此，本实验旨在通过对 Stable Diffusion 模型进行调参，提升其在古诗水墨画生成任务上的表现，实现个性化的艺术创作，同时高效利用计算资源，拓宽模型在跨文化艺术领域的应用边界。通过这一系列操作，参与者能够深入了解可控生成深度学习大模型的基本原理，诸如模型的训练机制、生成逻辑等关键技术。并且能够在实际操作中体验模型的生成效果，切实感受深度学习在跨领域艺术创作融合方面的魅力与潜力。

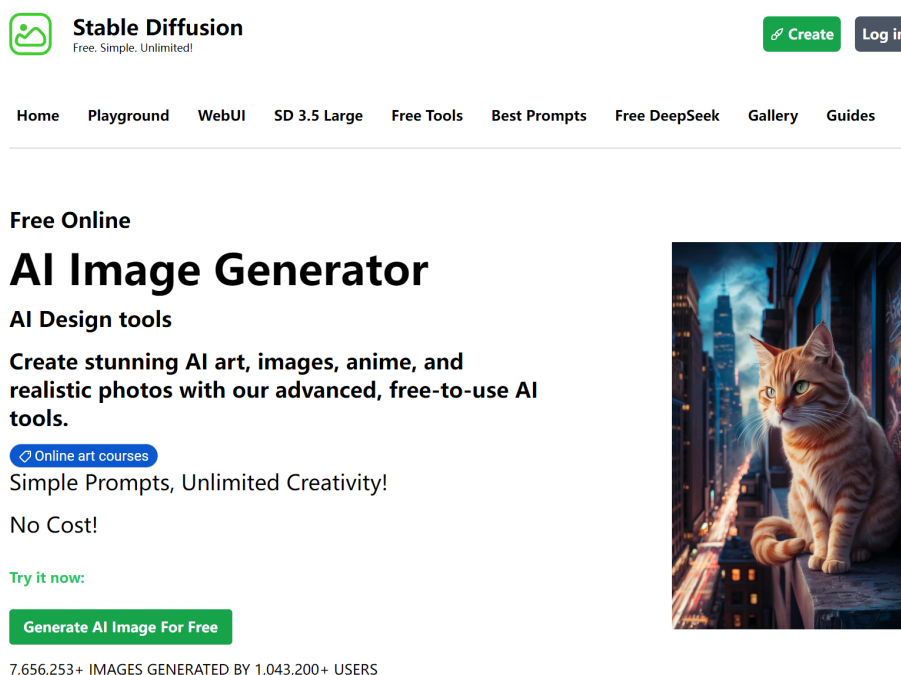


图 1 Stable Diffusion 模型在线应用（<https://stablediffusion.com/>）

②实验要求

实验要求包括使用 Stable Diffusion 大模型的代码与预训练参数来生成艺术图像，具体

要求为：首先利用现有代码和预训练参数进行基础图像生成；其次，结合实际需求自行训练并优化模型参数，以生成符合特定艺术风格或主题的图片（水墨画）；同时需详细记录实验过程中的参数调整、训练过程及生成结果。

另外，在前面的内容完成后，还可以训练模型生成写实画作，分析模型的实际效果，指出如何改进模型，进一步提升效果，或者自行换用其他可控生成模型，尝试不同模型的效果。

(2)解决问题的思路分析

在 Stable Diffusion (SD) 模型实验中，生成符合特定艺术风格的图像（如水墨画）是一个核心任务。该问题可抽象为利用 SD 大模型的预训练代码和参数进行基础图像生成，并结合实际需求通过自行训练优化模型参数以满足目标。在实现上，我计划使用现有预训练模型作为起点，记录参数调整过程，并通过实验数据验证生成效果，最终提交详细报告。主要解决思路包括基础生成、参数优化以及结果评估三个阶段。

基础生成的核心思路是基于 SD 模型的预训练权重和代码，直接生成初始艺术图像。该方法依赖现有资源，操作简单，能快速产出初步结果，但生成图像可能与目标风格偏差较大，仅适合初步探索和熟悉模型。

参数优化的核心思路是针对特定艺术风格，调整 SD 模型的超参数（如学习率、步数）及训练数据（特定风格的图像），通过迭代训练提升生成质量。结合实际输出与目标的对比，我采用手动调优和自动优化策略，逐步逼近理想效果。该方法能有效改进图像细节，适用于有明确风格需求的场景，也正是这次实验的要求。

最后进行结果评估，其核心思路是比较优化前后生成的图像，分析风格契合度及细节差异，总结参数调整的影响。结合实验记录，我计划主要由主观评判来评估模型性能，判断生成水墨画与原诗句意思是否对应，从而检验模型效果。该思路展现了从基础应用到深度优化的逐步演进，各具优势，满足本次 SD 实验的全面学习要求。

2.环境准备

为了训练和微调 Stable Diffusion 模型，由于模型体量较大，我需要借助 PyTorch 和 CUDA 进行训练。PyTorch 是一个基于 Python 的深度学习框架，它提供了灵活、易用的张量操作接口和强大的神经网络构建工具，可以非常轻松地定义自己的深度神经网络。CUDA (Compute Unified Device Architecture) 是 NVIDIA 开发的一种并行计算平台和编程模型，允许开发者利用 GPU 进行通用计算。通过 PyTorch 结合 CUDA，可以显著加速模型训练和推理，尤其在处理大规模数据或复杂网络时，效率远超 CPU，仅需在代码中启用 CUDA 支持（如 `torch.cuda.is_available()`）即可利用 GPU 资源。

由于下载的 stable-diffusion-v1-5 模型大小为 22G，对 GPU 的显存提出了较高要求，不适合在本地环境进行训练，于是我选用了 AutoDL 云平台的算力进行实验。我选用 NVIDIA vGPU-32GB，拥有 32G 显存，比较适合此实验使用。

镜像	PyTorch 2.5.1	Python 3.12(ubuntu22.04)	CUDA 12.4	更换
GPU	vGPU-32GB(32GB) * 1 升降配置			
CPU	12 vCPU Intel(R) Xeon(R) Platinum 8352V CPU @ 2.10GHz			
内存	90GB			

图 2 AutoDL 平台上云服务器配置

该实验在 Linux-5.4.0-155-generic-x86_64-with-glibc2.35 系统下进行，环境是 Ubuntu 22.02 LTS，包含 Python 3.12。在选用了 NVIDIA vGPU-32GB 的同时，该云服务器的 CPU 是 12 vCPU Intel(R) Xeon(R) Platinum 8352V CPU @ 2.10GHz，PyTorch 版本为 2.5.1，CUDA 版本为 12.4，都是比较新的版本。

3.模型设计

(1)模型原理

Stable Diffusion 是一种基于扩散模型（Diffusion Models, DMs）的生成式模型^[1]，广泛应用于图像生成任务（如文本生成图像、图像修复、图像风格转换等）。它具有高质量、开放、可调的特点。Stable Diffusion 的原理可以从扩散过程、反向生成过程、潜空间建模、条件控制等方面进行深入阐述。

扩散模型借鉴了非平衡热力学过程中的扩散现象：逐步将一个数据样本加入噪声直至近似纯高斯噪声，再训练模型学会反向去噪，从噪声中重构出原始样本。

①扩散模型的数学原理

A.正向扩散过程（Forward Process）

将原始图像 $\mathbf{x}_0$ 添加高斯噪声，逐步变为纯噪声 $\mathbf{x}_T$ 。该过程定义为马尔可夫链：

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$$

其中 β_t 为时间步 t 的噪声强度，逐步增大，通常是逐步增大的超参数序列（线性或余弦 schedule）。由于链式形式，我们可以直接从 $\mathbf{x}_0$ 推导任意步的结果：

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I}), \quad \bar{\alpha}_t = \prod_{s=1}^t \alpha_s = \prod_{s=1}^t (1 - \beta_s)$$

该方法简单，但需多步操作，仅为逆过程准备条件。

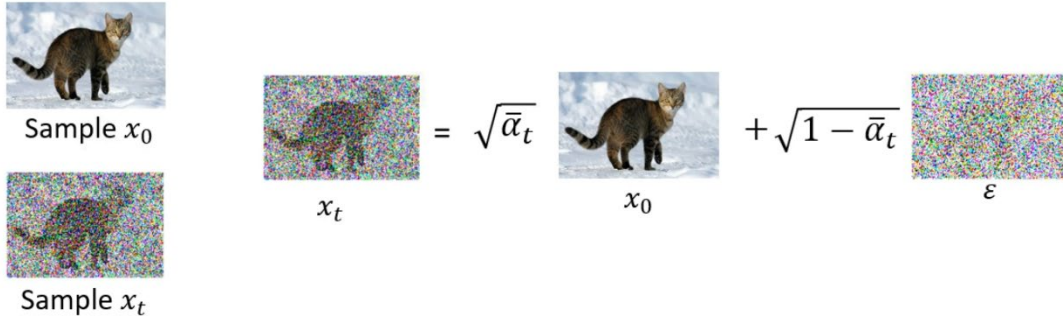


图 3 扩散过程示意图^[2]

B.反向生成过程（Reverse Process）

反向过程的目标是从纯噪声 $\mathbf{x}_T$ 开始，逐步去噪，恢复出图像 $\mathbf{x}_0$ ：

$$p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t) = N(\mathbf{x}_{t-1}; \boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t), \sum_{\theta}(\mathbf{x}_t, t))$$

由于真实反向分布 $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ 不可得，Stable Diffusion 训练一个神经网络 $\epsilon_{\theta}(\mathbf{x}_t, t)$ 来预测添加的噪声，从而估计出 $\mathbf{x}_0$ 。

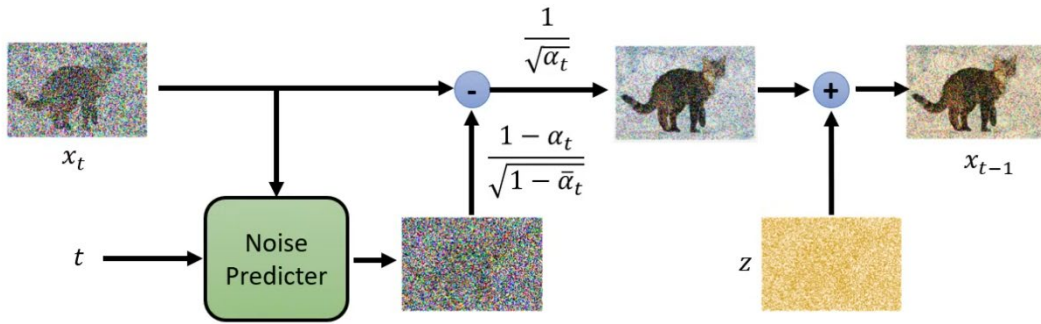


图 4 反向生成过程示意图^[2]

②训练目标

Stable Diffusion 模型训练目标是 최소화模型预测噪声与真实噪声之间的误差，常见的是：

$$L_{simple} = E_{\mathbf{x}_0, \epsilon, t} \left(\left\| \epsilon - \epsilon_{\theta}(\sqrt{\alpha_t} \mathbf{x}_0 + \sqrt{1 - \alpha_t} \epsilon, t) \right\|^2 \right)$$

这称为噪声预测目标，实际上等价于变分下界（ELBO）的简化形式。该目标也被称为 Denoising Score Matching。通过最小化预测噪声与真实噪声的均方误差，调整模型参数。该方法搜索效率高，能有效结合语义信息，适用于复杂艺术风格生成。

③潜空间生成（Latent Diffusion Model, LDM）

原始图像空间高维且消耗资源，Stable Diffusion 引入 VAE（变分自编码器）将图像压缩至潜空间 $\mathbf{z} = \mathcal{E}(\mathbf{x}_0)$ ，然后在潜空间中运行扩散过程：编码器 $\mathcal{E}$ 将图像压缩为潜空间，解码器 $\mathcal{D}$ 将潜空间映射回图像空间，同时在潜空间中训练一个扩散模型 $\epsilon_{\theta}(\mathbf{z}_t, t, c)$ ，在潜空间预测噪

声，显著降低计算开销。该方法资源需求低，生成效率高，适合复杂场景。

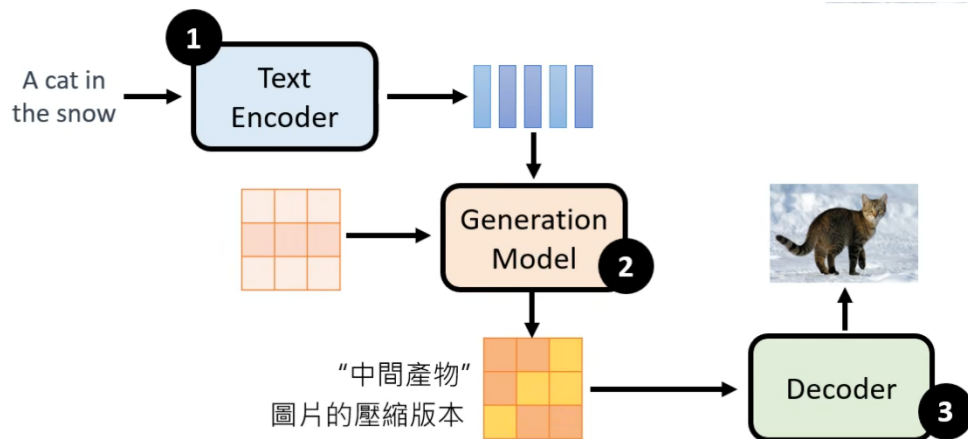


图 5 LDM 过程示意图^[2]

④条件控制（Text-to-Image）

Stable Diffusion 支持文本引导生成，即在训练中引入 CLIP 处理的文本编码 c ，条件扩散过程写作：

$$\epsilon_{\theta}(\mathbf{z}_t, t, c)$$

其中 c 是从输入文本提取的语义嵌入，通过预训练的 CLIP 模型将自然语言（如一句古诗）转换为与图像特征对齐的表示。在实现上，SD 利用交叉注意力（cross-attention）机制将文本特征动态注入 U-Net 网络的各个层级，增强模型对提示词的理解能力，从而提升文本与生成图像内容的对齐程度，同时 CLIP Score 可用于定量评估这一匹配度。

CLIP Score 的计算思路首先依赖 CLIP 的预训练能力，该模型在海量图像-文本对上训练，能够捕捉跨模态语义关系。其次，通过批量处理生成图像和多种提示词变体（如“a cat”与“a fluffy cat”），计算平均得分或置信区间，评估生成质量。

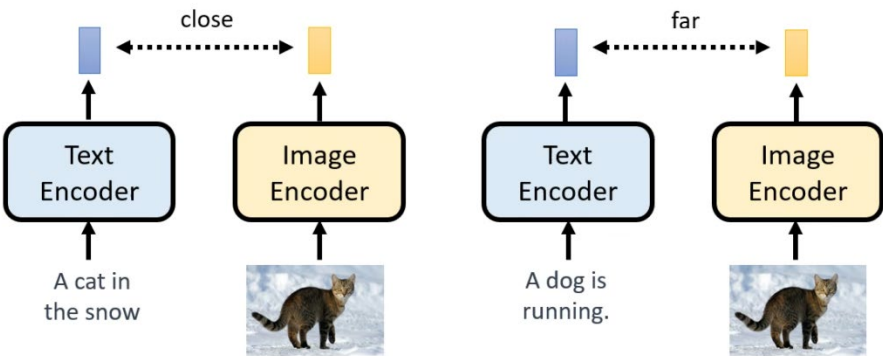


图 6 CLIP Score 计算思路示意图^[2]

具体优化在于，CLIP 的文本编码器将输入提示词映射为固定维度的向量 c ，结合时间步 t 指导 U-Net 预测潜空间噪声 $\mathbf{z}_t$ 中的去噪方向。交叉注意力机制通过查询（query）来自 U-

Net 的特征、键 (key) 和值 (value) 来自文本嵌入，计算加权和，动态调整特征表示，使生成过程更贴近文本语义。该方法搜索效率高，能有效捕捉复杂语义，但对 CLIP 质量和提示词设计依赖较大，仅适合有明确文本指导的场景。进一步扩展，条件控制还支持多模态输入，如结合图像条件 (Image-to-Image) 或掩码 (Inpainting)，通过调整 c 的来源或引入附加条件 (如边缘图)，实现图像修复或风格转换。

⑤采样过程 (Inference)

采样过程从高斯噪声 $\mathbf{z}_t \sim \mathcal{N}(0, \mathbf{I})$ 开始，通过如下方式反向去噪 T 步：

$$\mathbf{z}_{t-1} = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(\mathbf{z}_t - \frac{1 - \bar{\alpha}_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{z}_t, t, c) \right) + \sigma_t \epsilon, \epsilon \sim \mathcal{N}(0, \mathbf{I})$$

最后将 $\mathbf{z}_t$ 送入解码器 $\mathcal{D}$ 得到图像。。该方法步数可优化 (如 DDIM 减少至 50 步)，但计算开销较大，保证了生成质量。

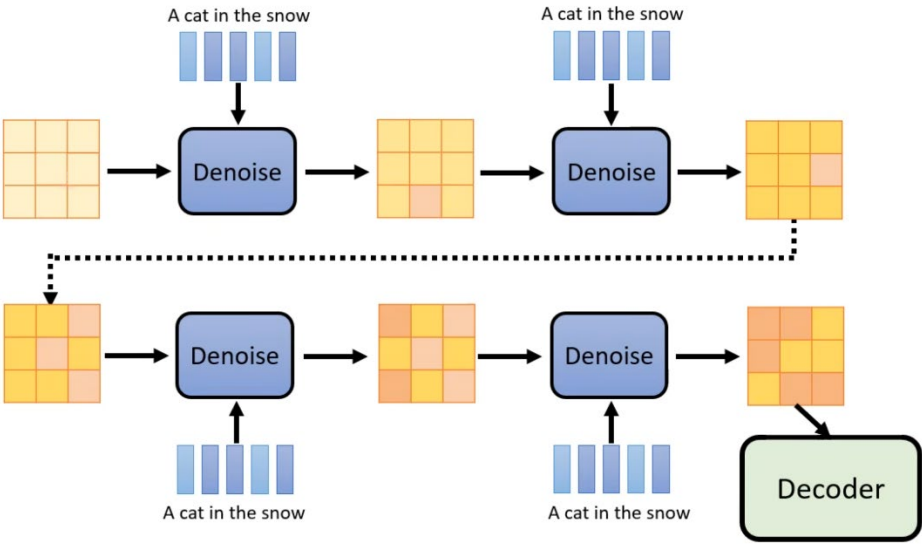


图 7 采样过程示意图^[2]

(2)模型结构

Stable Diffusion 是一种基于潜空间的扩散生成模型，其架构由 VAE、U-Net 和 CLIP 三部分组成。训练阶段，模型首先使用 VAE 编码器将图像压缩为潜变量，在潜空间中添加高斯噪声，并引入文本编码作为条件，利用 U-Net 网络预测噪声并最小化预测误差；推理阶段则从纯噪声出发，结合文本条件迭代去噪，还原出潜变量，最后通过 VAE 解码器生成图像，实现高效的文本到图像生成。

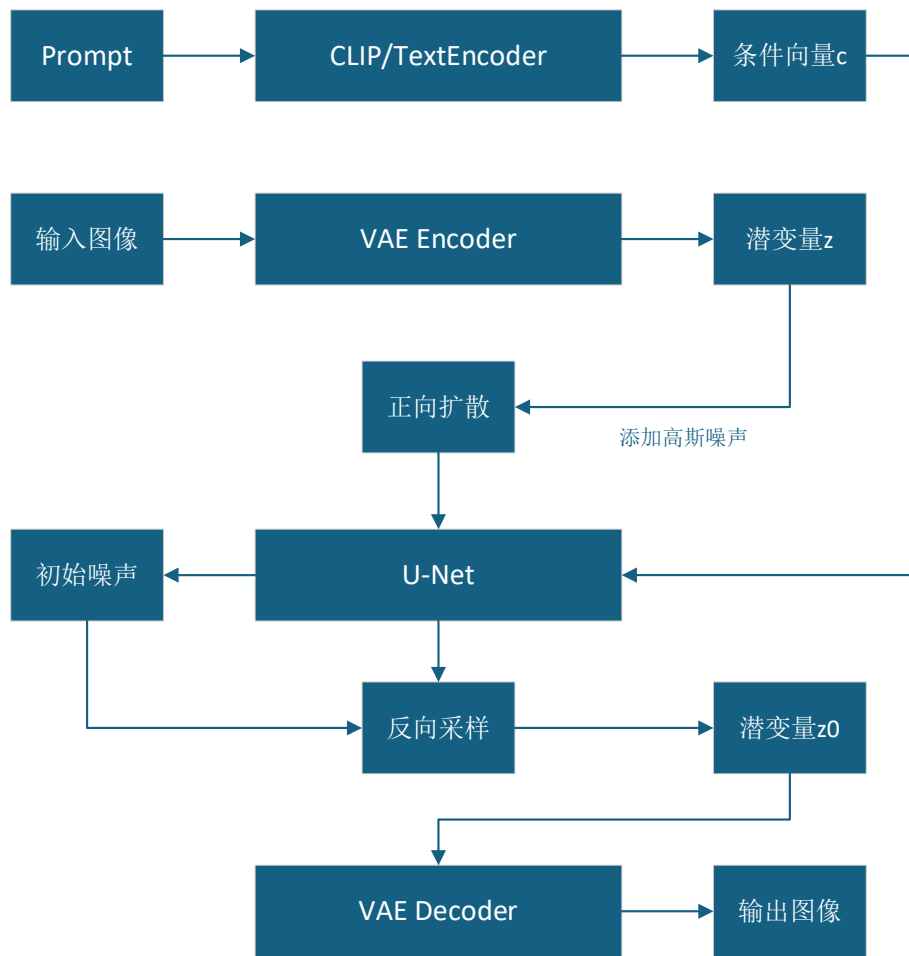


图 8 Stable Diffusion 模型结构图

(3)模型训练

环境设置：使用 Accelerate 框架配置分布式训练环境,支持混合精度训练和梯度累积。模型加载：加载预训练的 SD1.5 模型，包括 UNet、VAE、Text Encoder 等核心组件。数据处理：使用 Paint4Poem-Web-famous-subset 数据集,对诗词文本进行 tokenization，对图像进行尺寸调整和数据增强。训练流程：实现完整的训练循环,包括噪声添加、损失计算、梯度更新等，支持 EMA、gradient checkpointing 等优化技术。训练过程中定期保存检查点和生成验证图像。训练完成后保存最终模型,可用于推理生成。

①模型加载

在训练前先加载模型的核心组件。


```
noise_scheduler = DDPMScheduler.from_pretrained() # 噪声调度器
tokenizer = CLIPTokenizer.from_pretrained() # 文本标记器
text_encoder = CLIPTextModel.from_pretrained() # 文本编码器
vae = AutoencoderKL.from_pretrained() # 变分自编码器
unet = UNet2DConditionModel.from_pretrained() # U-Net 模型
```

②数据准备

使用 Paint4Poem-Web-famous-subset 作为训练数据集,数据包含诗词和对应图像的配对。在训练时,调整大小为 512x512,进行中心裁剪和随机翻转等增强,并使用 CLIP tokenizer 对诗词进行编码。

```
# 图像预处理
train_transforms = transforms.Compose([
    transforms.Resize(args.resolution),
    transforms.CenterCrop(args.resolution),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize([0.5], [0.5])
])
# 文本处理
tokenize_captions() # 对诗词进行标记化处理
```

③训练流程

整体训练流程包括初始化加载预训练模型（并冻结 VAE 和 Text Encoder）、训练循环、定期验证和定期保存检查点。

A.训练初始化

```
optimizer = AdamW(unet.parameters())
lr_scheduler = get_scheduler()
```

B.训练循环

a.从数据集获取图像诗词对

b.VAE 编码图像为 latents

VAE(变分自编码器)将输入图像编码为潜在空间表示(latents)。使用 `weight_dtype(fp16/bf16)` 进行混合精度训练,通过采样引入随机性,增强生成多样性,将 512x512 图像压缩为 64x64 潜在表示。VAE 编码是 SD 训练中的第一步,将高维图像压缩为低维 latents,为后续的扩散过程提供输入。

```
# VAE 编码图像

latents = vae.encode(images).latent_dist.sample()

latents = latents * vae.config.scaling_factor

# 文本编码

encoder_hidden_states = text_encoder(input_ids)[0]
```

c.添加随机噪声生成 noisy_latents

这个噪声添加过程是 Stable Diffusion 训练的核心部分,通过预测去噪过程来学习生成图像。使用噪声调度器将噪声添加到 latents,生成的 noisy_latents 将作为 U-Net 的输入,原始噪声或速度作为训练目标。

```
# 生成随机噪声

noise = torch.randn_like(latents)

# 添加噪声到潜空间

noisy_latents = noise_scheduler.add_noise(latents, noise, timesteps)
```

d.Text Encoder 编码诗词

Text Encoder(CLIP 文本编码器)负责将诗词文本转换为模型可理解的向量表示。输出维度为 `[batch_size, 77, 768]`, 77 为序列长度, 768 为隐层维度。Text Encoder 在训练中被冻结 (`requires_grad=False`),使用预训练权重,编码后的特征向量作为条件信息输入到 UNet 的 cross attention 层,编码过程如下:

原始诗词 -> Tokenizer 分词 -> Token ID 序列 -> Text Encoder -> 特征向量
春风又绿江南岸 -> [春,风,又,绿,江,南,岸] -> [1234,5678,...] -> [768维特征向量]

```
# 使用 CLIP text encoder 将 token 序列编码为特征向量

encoder_hidden_states = text_encoder(

    batch["input_ids"], # [batch_size, seq_len]

    return_dict=False

)[0] # [batch_size, seq_len, hidden_size]
```

e.U-Net 预测去噪过程

这个训练过程让 U-Net 学会了如何根据文本条件逐步去除噪声,从而能够生成符合描述的图像。

```
# UNet 预测噪声

model_pred = unet(noisy_latents, timesteps, encoder_hidden_states)
```

f.计算损失并优化

```
loss = F.mse_loss(model_pred, target) # 计算损失

accelerator.backward(loss) # 反向传播

accelerator.clip_grad_norm_(unet.parameters(), max_grad_norm) # 梯度裁剪

optimizer.step() # 优化器步进

lr_scheduler.step() # 学习率调整
```

4.模型实现

(1)数据处理模块 dataset_utils.py 中重要代码

```
def prepare_dataset(args, accelerator, tokenizer):
    """准备训练数据集

    Args:
        args: 包含数据集配置的命令行参数
        accelerator: 用于分布式训练的加速器对象
        tokenizer: 用于处理文本的分词器

    Returns:
        tuple: (训练数据集, 数据加载器)
    """
    加载数据集, 获取并验证数据集的列名, 确定图像列和标题列的名称, 验证列名是否存在。

    def tokenize_captions(examples, is_train=True):
```

```

    """将文本转换为 token ID
    Args:
        examples: 包含文本的样本
        is_train: 是否为训练模式（影响多个标题时的选择策略）
    """
    captions = [] 及其计算
    # 使用 tokenizer 处理文本
    inputs = tokenizer
    (
        captions, max_length=tokenizer.model_max_length,
padding="max_length", truncation=True, return_tensors="pt"
    )
    return inputs.input_ids

```

定义图像预处理转换。

```

def preprocess_train(examples):
    """预处理训练样本
    Args:
        examples: 包含图像 ID 的样本
    """
    加载并转换图像。
    # 应用图像转换并生成 token
    examples["pixel_values"] = [train_transforms(image) for image in
images]
    examples["input_ids"] = tokenize_captions(examples)
    return examples

```

使用 `accelerator` 确保在分布式环境中正确处理数据集。如果指定了最大样本数，随机选择样本。

```

def collate_fn(examples):
    """数据批次整理函数
    Args:
        examples: 批次中的样本列表
    Returns:
        dict: 包含处理后的像素值和 token ID
    """
    # 创建数据加载器
    train_dataloader = torch.utils.data.DataLoader(
        train_dataset,
        shuffle=True,
        collate_fn=collate_fn,
        batch_size=args.batch_size,
        num_workers=args.num_workers,
    )
    return train_dataset, train_dataloader

```

(2)模型加载模块 `model_utils.py` 中重要代码

```

def load_all_of_models_schedulers_and_others(args):
    """加载 Stable Diffusion 训练所需的所有模型组件
    该函数加载以下组件：
    1. 噪声调度器(DDPM) - 用于添加和移除噪声
    2. 分词器(CLIP) - 用于文本处理
    3. 文本编码器(CLIP) - 用于生成文本嵌入
    4. VAE - 用于图像编码和解码
    5. UNet - 核心扩散模型
    6. EMA UNet - 模型参数的指数移动平均版本（可选）
    Args:
        args: 包含模型配置的命令行参数对象
    Returns:
        tuple: 返回加载的所有模型组件
            - noise_scheduler: 噪声调度器
            - tokenizer: CLIP 分词器
            - text_encoder: CLIP 文本编码器
            - vae: 变分自编码器
            - unet: U-Net 模型
            - ema_unet: EMA 版本的 U-Net 模型（如果启用）
    """
    # 加载噪声调度器
    noise_scheduler = DDPMScheduler.from_pretrained(args.model_path,
subfolder="scheduler")
    # 加载 CLIP 分词器
    tokenizer = CLIPTokenizer.from_pretrained(
        args.model_path, subfolder="tokenizer",
revision=args.model_revision
    )
    # 创建 DeepSpeed Zero 优化上下文管理器
    def deepspeed_ban_init():
        """创建 DeepSpeed Zero 初始化的禁用上下文
        用于临时禁用 DeepSpeed Zero 优化，确保大模型加载正确
        """
        deepspeed_plugin = AcceleratorState().deepspeed_plugin if
accelerate.state.is_initialized() else None
        return [] if deepspeed_plugin is None else
[deepspeed_plugin.zero3_init_context_manager(enable=False)]
    # 在禁用 DeepSpeed Zero 优化的上下文中加载大型模型
    with ContextManagers(deepspeed_ban_init()):
        # 加载 CLIP 文本编码器
        text_encoder = CLIPTextModel.from_pretrained(
            args.model_path, subfolder="text_encoder",
revision=args.model_revision, variant=args.model_variant
        )

```

```

        # 加载 VAE 模型
        vae = AutoencoderKL.from_pretrained(
            args.model_path, subfolder="vae",
revision=args.model_revision, _variant=args.model_variant
        )
        # 加载 U-Net 模型
        unet = UNet2DConditionModel.from_pretrained(
            args.model_path, subfolder="unet",
revision=args.non_ema_model_revision
        )
        # 冻结 VAE 和文本编码器的参数
        vae.requires_grad_(False)
        text_encoder.requires_grad_(False)
        # 设置 U-Net 为训练模式
        unet.train()
        # 可选: 创建 EMA 版本的 U-Net
        ema_unet = None
        if args.ema:
            ema_unet = UNet2DConditionModel.from_pretrained(
                args.model_path, subfolder="unet",
model_revision=args.model_revision, model_variant=args.model_variant
            )
            # 将 U-Net 封装在 EMA 模型中
            ema_unet = EMAModel(ema_unet.parameters(),
model_cls=UNet2DConditionModel, model_config=ema_unet.config)
        return noise_scheduler, tokenizer, text_encoder, vae, unet, ema_unet

```

(3)初始化模块 `utils.py` 中重要代码

```

def initialize_accelerator(args):
    """
    初始化 Accelerate 对象，用于分布式训练和混合精度设置。
    参数:
        args: 命令行参数，包含训练配置
    返回:
        accelerator: 配置好的 Accelerate 对象
    """
    logging_direction = os.path.join(args.output_direction,
args.logging_direction)
    accelerator_project_config =
ProjectConfiguration(project_dir=args.output_direction,
logging_dir=logging_direction)
    云平台日志记录。
    logger.info(accelerator.state, main_process_only=False)
    return accelerator

```

(4)训练模块 `utils.py` 中重要代码

①设置优化器和学习率调度器的函数

```
def setup_optimizer_and_scheduler(args, unet, train_dataloader,
accelerator):
    """设置优化器和学习率调度器
    Args:
        args: 包含训练配置的参数对象
        unet: 需要优化的 UNet 模型
        train_dataloader: 训练数据加载器
        accelerator: 分布式训练加速器
    Returns:
        tuple: (optimizer, lr_scheduler) - 优化器和学习率调度器
    """
    如果启用学习率缩放，根据批次大小和设备数调整学习率。
    # 创建 AdamW 优化器
    optimizer_cls = torch.optim.AdamW
    optimizer = optimizer_cls(
        unet.parameters(),
        lr=args.learning_rate,
        betas=(args.adamb1, args.adamb2), # Adam 优化器的 beta 参数
        weight_decay=args.adam_decay, # 权重衰减率
        eps=args.adameps, # epsilon 值防止除零
    )
    计算每轮的更新步数。
    # 创建学习率调度器
    scheduler = get_scheduler(
        args.scheduler, # 调度器类型
        optimizer=optimizer,
        num_warmup_steps=args.warmup_steps *
accelerator.num_processes, # 预热步数
        num_training_steps=args.max_steps *
accelerator.num_processes, # 总训练步数
    )
    return optimizer, scheduler
```

②验证模型的函数

```
def val_model(vae, text_encoder, tokenizer, unet, args, accelerator,
weight_dtype, epoch):
    """执行验证并生成示例图像
    Args:
        vae: 变分自编码器用于图像编码/解码
        text_encoder: 文本编码器用于处理提示词
        tokenizer: 分词器用于处理文本输入
```



```

    unet: UNet 模型
    args: 训练参数配置
    accelerator: 分布式训练加速器
    weight_dtype: 模型权重的数据类型
    epoch: 当前训练轮次
Returns:
    list: 生成的验证图像列表
"""
logger.info("正在运行验证...")
# 构建 Stable Diffusion 推理管线
pipeline = StableDiffusionPipeline.from_pretrained(
    args.model_path,
    vae=accelerator.unwrap_model(vae),
    text_encoder=accelerator.unwrap_model(text_encoder),
    tokenizer=tokenizer,
    unet=accelerator.unwrap_model(unet),
    safety_checker=None, # 禁用安全检查器以提高速度
    model_revision=args.model_revision,
    model_variant=args.model_variant,
    torch_dtype=weight_dtype,
)
# 配置推理设备和优化
pipeline = pipeline.to(accelerator.device)
pipeline.set_progress_bar_config(disable=True) # 禁用进度条
if args.xformers:
    pipeline.xformers() # 启用 xformers 优化
    设置随机数生成器。
images = [] # 用于存储生成的图像
# 创建验证图像保存目录
if accelerator.is_main_process:
    validation_dir = os.path.join(args.output_direction,
"validation_images")
    os.makedirs(validation_dir, exist_ok=True)
# 为每个验证提示词生成图像
for i in range(len(args.validation_prompts)):
    处理 MPS 设备的特殊情况。
    # 生成图像
    with autocast_ctx:
        image = pipeline(args.validation_prompts[i],
num_inference_steps=20, generator=generator).images[0]
        保存图像。
del pipeline
torch.cuda.empty_cache()
return images

```

③执行训练的函数

```
def train_loop(args, accelerator, unet, ema_unet, vae, text_encoder,
tokenizer, train_dataset, train_dataloader, optimizer, scheduler,
noise_scheduler):
    """执行 Stable Diffusion 模型的训练循环
    该函数实现了完整的训练流程，包括：
    1. 模型和优化器的设备分配与初始化
    2. 训练状态的恢复(如果需要)
    3. 训练循环的执行
    4. 损失计算和优化
    5. 检查点保存
    6. 验证和日志记录
    Args:
        args: 包含所有训练配置的参数对象
        accelerator: 分布式训练加速器对象
        unet: 主要训练的 U-Net 模型
        ema_unet: EMA 版本的 U-Net 模型(可选)
        vae: 用于图像编码/解码的 VAE 模型
        text_encoder: 用于文本编码的 CLIP 模型
        tokenizer: 用于文本处理的分词器
        train_dataset: 训练数据集
        train_dataloader: 训练数据加载器
        optimizer: 优化器(通常是 AdamW)
        scheduler: 学习率调度器
        noise_scheduler: 噪声调度器，用于扩散过程
    Returns:
        tuple: (global_step, first_epoch) - 总训练步数和起始轮次
    """
    # 1. 模型优化配置
    if args.grad_checkpoint:
        unet.enable_gradient_checkpointing() # 启用梯度检查点以节省显存
    if args.tf32:
        torch.backends.cuda.matmul.allow_tf32 = True # 启用 TF32 加速计算
    # 2. 准备模型和优化器
    unet, optimizer, train_dataloader, scheduler = accelerator.prepare(
        unet, optimizer, train_dataloader, scheduler
    )
    if args.ema:
        ema_unet.to(accelerator.device) # 将 EMA 模型移到正确设备
    # 3. 设置模型精度
    weight_dtype = torch.float32
    if accelerator.mixed_precision == "fp16":
        weight_dtype = torch.float16
        args.precision = accelerator.mixed_precision
```

```

elif accelerator.mixed_precision == "bf16":
    weight_dtype = torch.bfloat16
    args.precision = accelerator.mixed_precision
# 4. 将其他模型转移到目标设备和精度
text_encoder.to(accelerator.device, dtype=weight_dtype)
vae.to(accelerator.device, dtype=weight_dtype)
# 5. 计算训练步数和轮次
# 6. 初始化训练追踪器
if accelerator.is_main_process:
    tracker_config = dict(vars(args))
    tracker_config.pop("validation_prompts")
    accelerator.init_trackers(args.project_name, tracker_config)
# 7. 辅助函数：解包装模型
def unwrap_model(model):
    model = accelerator.unwrap_model(model)
    model = model._orig_mod if is_compiled_module(model) else model
    return model
# 8. 计算总批次大小并打印训练信息
# 9. 初始化或恢复训练状态
global_step = 0
first_epoch = 0
if args.resume_ckpt: # 如果需要从检查点恢复
    if args.resume_ckpt != "latest":
        path = os.path.basename(args.resume_ckpt)
    else:
        找到最新的检查点。
    if path is None:
        args.resume_ckpt = None
        initial_global_step = 0
    else:
        # 加载检查点状态
else:
    initial_global_step = 0
# 10. 创建进度条
# 11. 主训练循环
for epoch in range(first_epoch, args.epochs):
    train_loss = 0.0
    # 12. 批次训练循环
    for step, batch in enumerate(train_dataloader):
        if batch is None:
            continue
        # 13. 梯度累积上下文
        with accelerator.accumulate(unet):
            # 13.1 获取图像潜在表示

```

```

        latents =
vae.encode(batch["pixel_values"].to(weight_dtype)).latent_dist.sample()
        latents = latents * vae.config.scaling_factor
        # 13.2 生成和处理噪声
        noise = torch.randn_like(latents)
        if args.noise_shift: # 应用噪声偏移
            noise += args.noise_shift * torch.randn(
                (latents.shape[0], latents.shape[1], 1, 1),
device=latents.device
            )
        if args.input_noise: # 应用输入噪声
            new_noise = noise + args.input_noise *
torch.randn_like(noise)
        # 13.3 采样时间步并添加噪声
        bsz = latents.shape[0]
        timesteps = torch.randint(0,
noise_scheduler.config.num_train_timesteps, (bsz,),
device=latents.device)
        timesteps = timesteps.long()
        noisy_latents = noise_scheduler.add_noise(latents,
new_noise if args.input_noise else noise, timesteps)
        # 13.4 获取文本编码
        encoder_hidden_states = text_encoder(batch["input_ids"],
return_dict=False)[0]
        # 13.5 设置预测类型
        # 13.6 确定训练目标
        # 13.7 应用 DREAM 训练方法(如果启用)
        # 13.8 模型预测
        model_pred = unet(noisy_latents, timesteps,
encoder_hidden_states, return_dict=False)[0]
        # 13.9 计算损失
        if args.snr_weight is None:
            # 普通 MSE 损失
            loss = F.mse_loss(model_pred.float(),
target.float(), reduction="mean")
        else:
            # 带 SNR 权重的损失
            snr = compute_snr(noise_scheduler, timesteps)
            mse_loss_weights = torch.stack([snr, args.snr_weight
* torch.ones_like(timesteps)], dim=1).min(dim=1)[0]
            if 预测类型 == "epsilon":
                mse_loss_weights = mse_loss_weights / snr
            elif 预测类型 == "v_prediction":
                mse_loss_weights = mse_loss_weights / (snr + 1)

```

```

        loss = F.mse_loss(model_pred.float(),
target.float(), reduction="none")
        loss = loss.mean(dim=list(range(1,
len(loss.shape)))) * mse_loss_weights
        loss = loss.mean()
        # 13.10 处理损失
        avg_loss =
accelerator.gather(loss.repeat(args.batch_size)).mean()
        train_loss += avg_loss.item() / args.grad_accum_steps
        # 13.11 反向传播和优化
        accelerator.backward(loss)
        if accelerator.sync_gradients:
            accelerator.clip_grad_norm_(unet.parameters(),
args.grad_norm)
        optimizer.step()
        scheduler.step()
        optimizer.zero_grad()
    # 14. 同步后的操作
    if accelerator.sync_gradients:
        if args.ema:
            ema_unet.step(unet.parameters())
        progress_bar.update(1)
        global_step += 1
        # 14.1 记录训练指标
        swanlab.log({"训练损失": train_loss}, step=global_step)
        swanlab.log({"学习率": scheduler.get_last_lr()[0]},
step=global_step)
        train_loss = 0.0
        # 14.2 保存检查点
    # 15. 更新进度条
    # 16. 检查是否达到最大步数
    # 17. 执行验证(如果配置)
    if accelerator.is_main_process:
        if args.validation_prompts is not None and epoch %
args.validation_epochs == 0:
            if args.ema:
                ema_unet.store(unet.parameters())
                ema_unet.copy_to(unet.parameters())
            val_model(
                vae,
                text_encoder,
                tokenizer,
                unet,
                args,

```

```

        accelerator,
        weight_dtype,
        global_step,
    )
    if args.ema:
        ema_unet.restore(unet.parameters())
    return global_step, first_epoch

```

其余辅助代码不再一一呈现。

5.实验结果

在训练过程中，由于训练集较大，进行了 10 个 epoch，Training Steps 为 15000，batch size 大小为 1，Gradient Accumulation Steps 为 4，共用时 5h35min。最终执行结果如下：

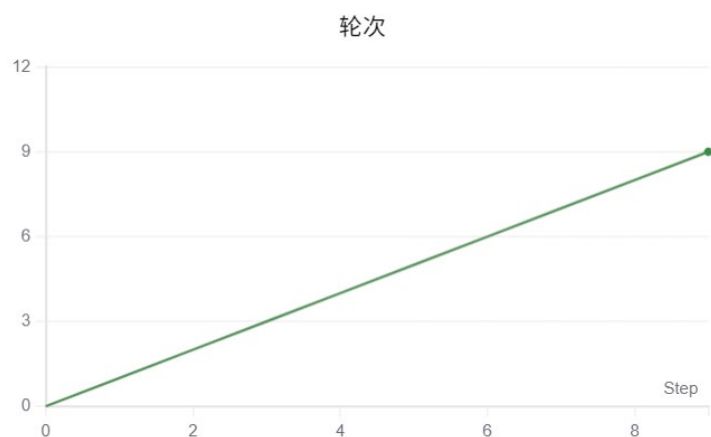


图 9 Stable Diffusion 模型训练 epoch 图



图 10 Stable Diffusion 模型训练 loss 图

在训练过程中，每过一个 epoch，模型会输出 1 组图片，观察输出前后，可以观察到模型性能上的提升。

Prompt 1 山气日夕佳，飞鸟相与还。

由于 epoch 较多，这里只展示有代表性的和发生显著变化的，不再一一列举。



Epoch 1



Epoch 2



Epoch 6



Epoch 9

validation_image



最终结果

山气日夕佳，飞鸟相与还。

Prompt 2 春眠不觉晓，处处闻啼鸟。

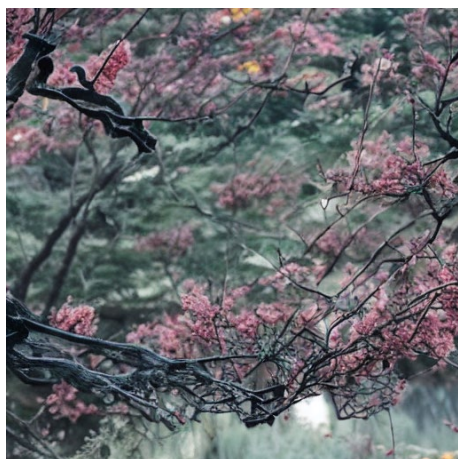
由于 epoch 较多，这里只展示有代表性的和发生显著变化的，不再一一列举。



Epoch 1



Epoch 2



Epoch 6



Epoch 9

validation_image



最终结果

春眠不觉晓，处处闻啼鸟。

可以看出，随着训练轮次的增加，效果越来越往水墨画风格变化，Prompt 1 飞鸟的元素

也逐渐明显，Prompt 2 出现房屋的元素，更加符合意境。幻觉（如楼房、不合适的花朵等）也逐渐消失，训练效果较好。限于篇幅，其余实验结果在下面一同放出：

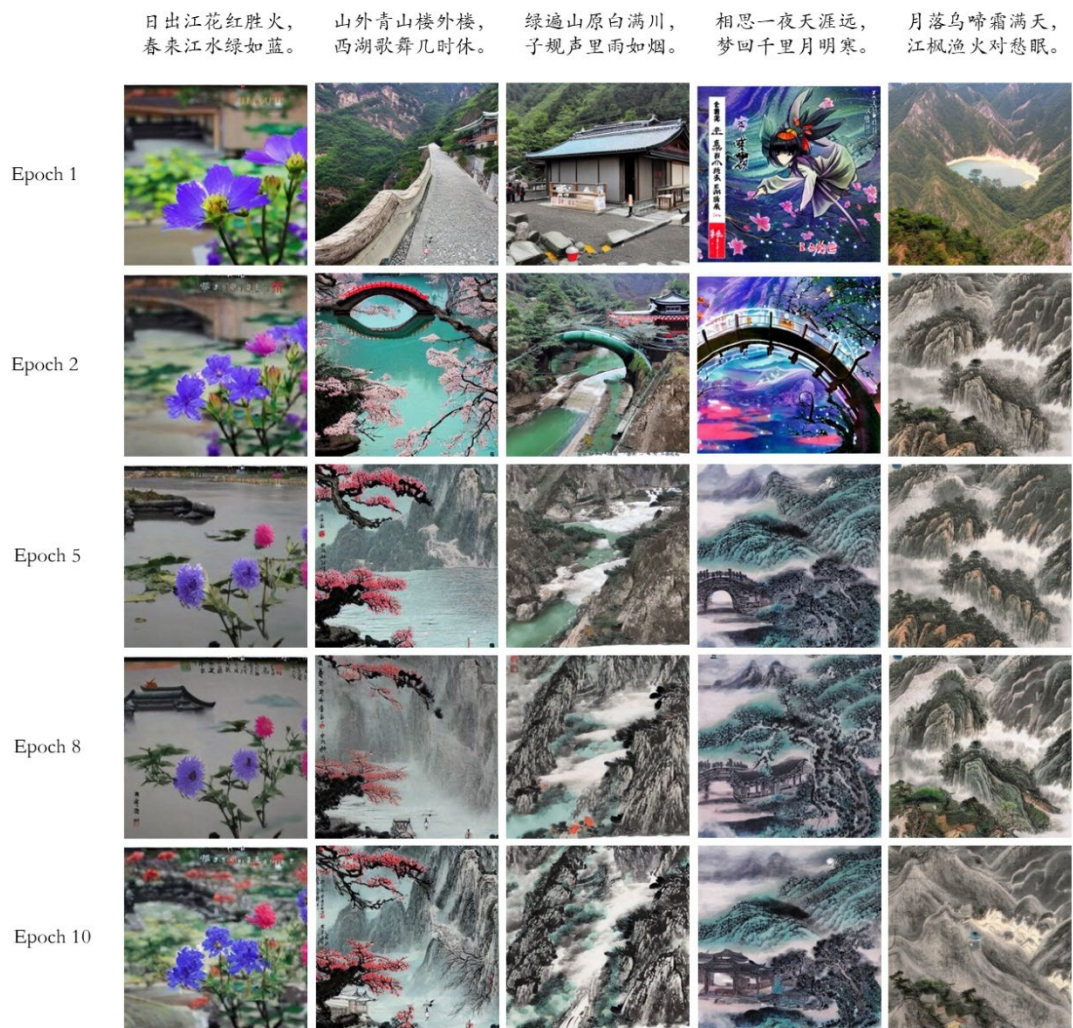


图 11 Stable Diffusion 模型训练过程效果展示图

6.总结与分析

(1)对模型的理解和评价

Stable Diffusion 是一种基于“扩散模型”的图像生成方法，它在训练时通过向图像添加噪声、学习如何去噪的过程实现图像的再构建。与传统扩散模型不同，Stable Diffusion 首先使用 VAE（变分自编码器）将图像压缩到一个低维潜在空间中，再在这个潜在空间中进行扩散与去噪，从而在保持生成效果的同时大幅减少计算成本。文本信息通过 CLIP 模型编码后作为条件输入，引导图像的生成方向。

Stable Diffusion 的最大技术优势在于“潜在空间扩散”，使得图像生成变得高效且资源消耗低；相比像素空间扩散的模型，它更易于部署在消费级设备上。其结构采用 U-Net 作为去噪主干网络，结合文本提示（Prompt）提供灵活的控制能力，既可生成高分辨率图像，也

能支持多种后续模型微调技术，如 LoRA 和 DreamBooth，极大扩展了模型的适用性和二次开发能力。

在实际应用中，Stable Diffusion 已广泛应用于艺术创作、商业广告、游戏设计、时尚搭配等多个场景。用户可以通过简单的文本输入快速生成高度定制的视觉内容，降低了美术创作门槛，极大激发了创意产业的生产力。同时，该模型也在教育、科研、医疗图像合成等领域展现出广阔的潜力，成为 AIGC（生成式人工智能）浪潮中的关键技术之一。

尽管 Stable Diffusion 拥有强大的生成能力，但它也存在诸多限制。例如，模型在处理人手、文字等结构复杂的内容时仍容易出错，生成图像中常见畸形或逻辑不一致的问题。此外，它的文本理解能力有限，有时会对 Prompt 理解不准确，输出偏离预期。尤其是面对中文 Prompt，常常无法正确理解。

未来，Stable Diffusion 的发展方向可能集中在可控性增强、多模态生成、模型可解释性和安全性四大领域。通过引入如 ControlNet^[3]、T2I-Adapter^[4]等技术，可以实现更细致的图像控制。与语音、视频、3D 建模等模态的融合将推动其应用边界拓展。

(2)训练和调试中遇到的问题

①训练集污染的问题

Paint4Poem-Web-famous-subset 数据集存在较严重的污染，第一个就是图片格式未对齐，存在 jpeg 和 png 两个格式，我通过代码全部转换成了 jpeg；其次，训练集中还有少量损坏的图片，我在训练过程中跳过了这些图片。

另外，该数据集不太适用于古诗转水墨画任务，第一个原因是训练集提示词的不合理，或是过于晦涩，或是没有画面感，示例如下：

image_id	poem
img_83052	日往菲薇，月来扶疏。
img_83053	雷雨窈冥而未半，皦日笼光於绮寮。
img_83057	人皆可以为尧舜。

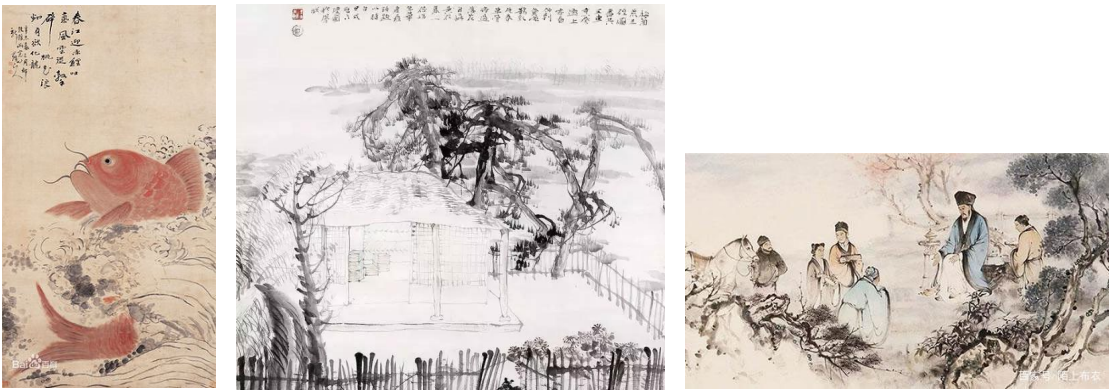


图 12 img_83052、img_83053 和 img_83057 训练集图片

这些图也无法与相应训练用的提示词对应，为训练模型带来了困难。同时训练集中还存在不是水墨风格的，也为微调带来了困难。示例如下：



图 13 img_89064.jpeg 训练集图片（提示词为“军中宜剑舞，塞上重笳音”）

②汉语古诗 Prompt 效果不佳的问题

在模型检验过程中，使用古诗作为 Prompt 常常使 Stable Diffusion 模型无法捕捉到是图中的所有意象，造成生成图像效果不佳。于是我尝试了使用古诗、翻译成现代汉语、翻译成英语三种 Prompt，分别进行试验，结果如下：

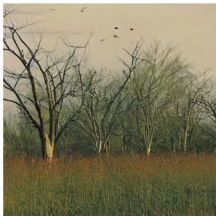
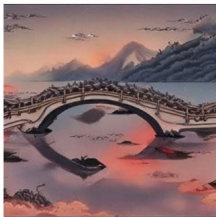
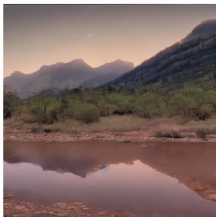
	古诗	现代汉语	英语
南朝四百八十寺， 多少楼台烟雨中。			
海上生明月， 天涯共此时。			
草长莺飞二月天， 拂堤杨柳醉春烟。			
江流天地外， 山色有无中。			

图 14 Stable Diffusion 模型不同语言的 Prompt 效果对比图

可以看出，不同语言的 Prompt 效果确实不同，不用古诗会失去微调后带来的水墨画的风格，生成图像可能会意象更准确。

(3)模型改进

①写实风格画作的生成

写实风格化作生成的第 1 种方式是换用现代汉语或英语的 Prompt，如上图（图 14）所示，可以生成较为写实的画作。

另外，我还尝试用新的数据集对模型进行微调，我一开始准备使用 LAION-5B/LAION-Aesthetics，这是一个超大规模开放图文配对数据集，Stable Diffusion 原始模型就是在 LAION-2B 和 LAION-5B 上训练的。包括了大量写实风格图像，覆盖人物、自然、建筑、商品等，且有审美评分子集（如 LAION-Aesthetics）。但因为数据量极大，需具备强算力及过滤工具，无法满足。于是我改用 COCO (Common Objects in Context, <https://cocodataset.org>)，一个经典的计算机视觉数据集，包含丰富的生活场景图像与精准标注。图像真实、描述详细，适合训练文本引导的写实生成模型。而且数据量不大，适合微调或预训练补充。

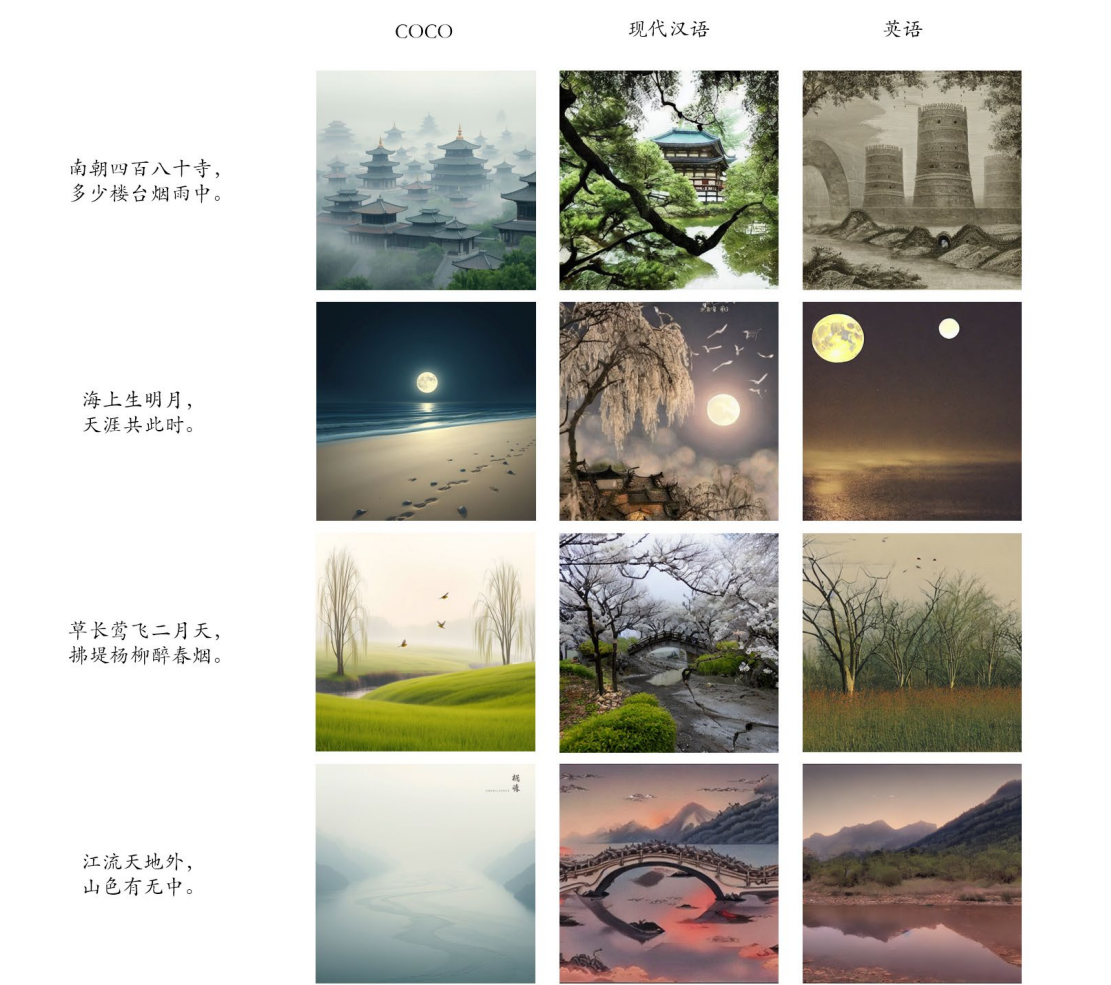


图 15 Stable Diffusion 模型微调不同数据集效果对比图

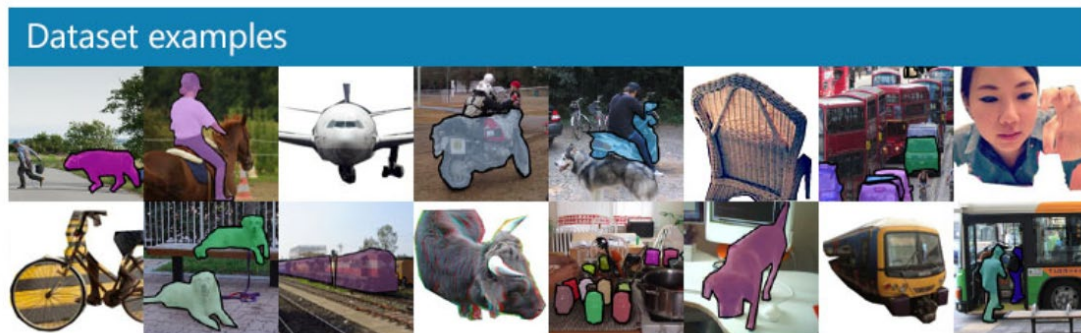


图 16 COCO 数据集官方示例图

可以看出（图 15），换用 COCO 数据集之后，模型可以生成写实图像，且写实效果比仅换 Prompt 更好。

②训练方式的改进

为了达到更好的训练效果，我对训练方式了改进，具体体现在数据处理优化、启用 DREAM 训练、使用 SNR 加权损失、验证和检查点策略的优化等。

数据处理优化主要体现在数据增强上，增强模型的鲁棒性和泛化能力：

```
--center_crop \           # 中心裁剪
--random_flip \          # 随机翻转
--image_size=512         # 保持高分辨率
```

DREAM(Dynamic REplacement And Modification)是一种改进的 Stable Diffusion 训练方法，旨在提高训练效率和生成质量。这是一个动态更新过程，使用当前模型预测来更新训练样本，在训练过程中不断优化噪声分布。模型进行自适应学习，逐步学习更复杂的特征，避免过度拟合简单模式，在创新性和保真度之间取得平衡。

```
noisy_latents, target = compute_dream_and_update_latents(
    unet,
    noise_scheduler,
    timesteps,
    noise,
    noisy_latents,
    target,
    encoder_hidden_states,
    args.dream_preservation,
)
```

加上这个方法后，生成图像保真明显提高，示例如下：

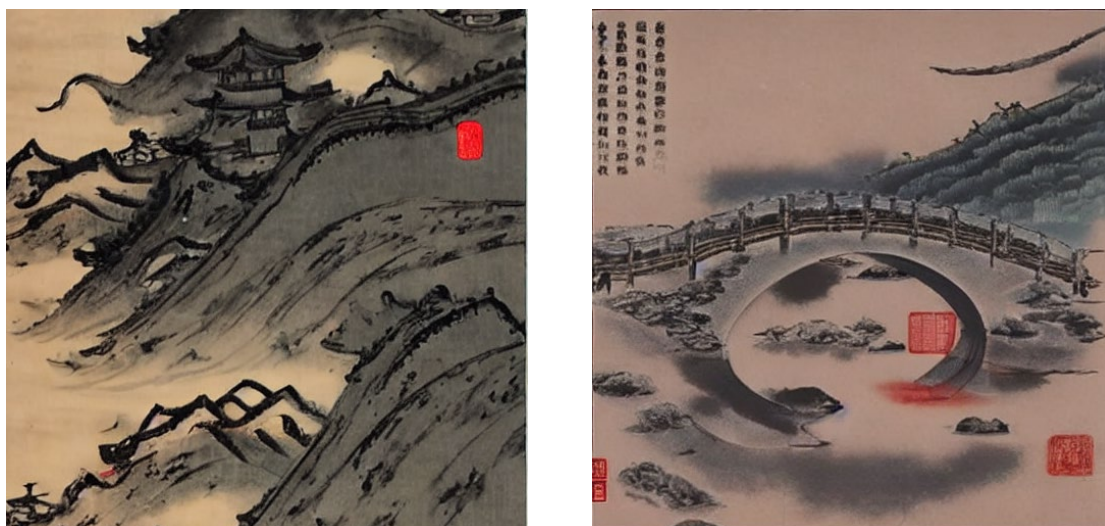


图 17 Stable Diffusion 模型加上 DREAM 后的效果图

我还改用渐进式训练，提高训练效率。

```
# 第一阶段：稳定化训练
--max_steps=1000 \
--learning_rate=1e-6
# 第二阶段：主要训练
--max_steps=10000 \
--learning_rate=2e-6
# 第三阶段：微调
--max_steps=2000 \
--learning_rate=5e-7
```

(4)其他模型尝试

我尝试换用 FLUX.1-schnell 模型生成图像，这是由 Hugging Face 上的组织 black-forest-labs 发布的一个用于快速推理的 Stable Diffusion 风格模型^[5]。调用方式简写如下：

```
pipe = FluxPipeline.from_pretrained("black-forest-labs/FLUX.1-schnell",
torch_dtype=torch.bfloat16)
pipe.enable_model_cpu_offload()
prompt = "xxxxxx"
image = pipe(
    prompt,
    guidance_scale=6.0,
    num_inference_steps=4,
    max_sequence_length=256,
    generator=torch.Generator("cpu").manual_seed(0)
).images[0]
image.save("flux-schnell.png")
```

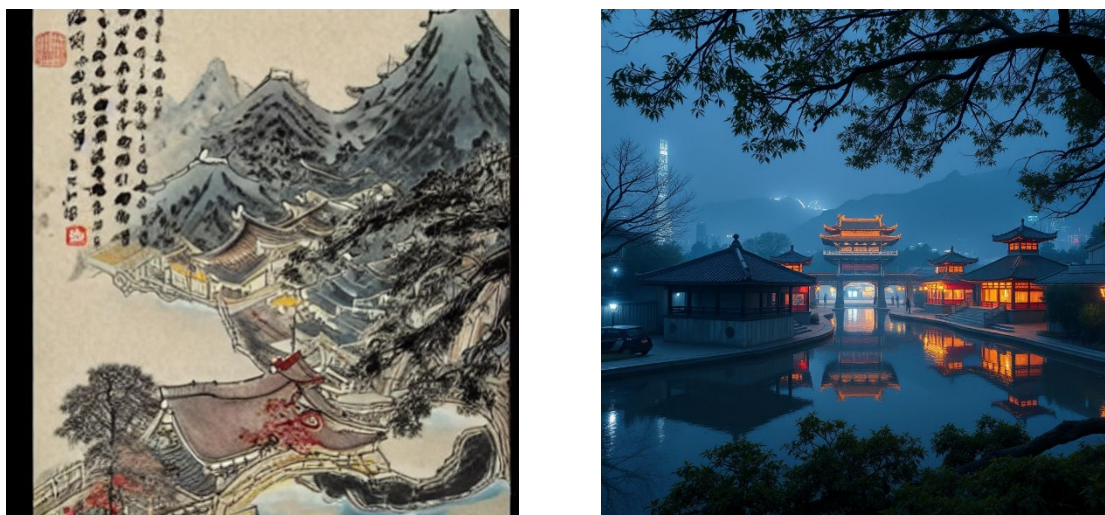


图 18 微调 Stable Diffusion 模型与 Flux 模型的对比图

可以看出，本地微调的 SD 模型生成古诗图像的水墨画风格更好，更好的完成了此次实验的任务。

(5)实验收获

在本次 Stable Diffusion 模型的实验过程中，我主要收获了以下几点：首先，对扩散模型的核心原理有了更深入理解，特别是其通过逐步加噪和反向去噪实现图像生成的机制，提升了我对生成模型在概率建模方面的整体认知；其次，实际操作中接触了 VAE 编码器、U-Net 网络结构以及 CLIP 文本嵌入的协同工作流程，增强了我在多模块协同建模方面的实践能力；此外，在模型训练和推理阶段，通过调试噪声调度器、条件控制机制等关键参数，我体会到了如何在保留生成质量的同时优化模型效率。整体而言，该实验不仅加深了我对扩散式生成模型的理论认知，也提升了实际部署与调优能力。。

7. 参考文献及资料

- [1] Rombach R, Blattmann A, Lorenz D, et al. High-Resolution Image Synthesis with Latent Diffusion Models[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2022: 10684-10695.
- [2] Hung-yi Lee, Generative AI (2024 Spring), <https://speech.ee.ntu.edu.tw/~hylee/genai/2024-spring.php>.
- [3] Zhang L, Rao A, Agrawala M. Adding Conditional Control to Text-to-Image Diffusion Models[C]//Proceedings of the IEEE/CVF International Conference on Computer Vision. 2023: 3836-3847.
- [4] Mou C, Wang X, Xie L, et al. T2i-adapter: Learning Adapters to Dig out More Controllable Ability for Text-to-Image Diffusion Models[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2024, 38(5): 4296-4304.
- [5] black-forest-labs. FLUX.1-schnell. <https://huggingface.co/black-forest-labs/FLUX.1-schnell>.